

Tarea 3 – Simulación estocástica

Lenin Pavón Alvarez

2026-05-27

Para llevar a cabo la ejecución de este proyecto se requiere de la importación de las siguientes bibliotecas

- `ggplot2`
- `tidyverse`

Y establecemos la semilla 42.

```
library(ggplot2)
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v lubridate  1.9.5      v tibble    3.3.1
## v purrr      1.2.2      v tidyr     1.3.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

set.seed(42)
```

Problema 1. Metrópolis-Hastings

Reactivo 1.1

Sea $\pi = (0.1, 0.2, 0.3, 0.4)$ la distribución objetivo, con soporte en el conjunto $\{1, 2, 3, 4\}$

Ahora definimos a la distribución objetivo con su soporte

```
obj <- c(0.1, 0.2, 0.3, 0.4)
sop <- c(1, 2, 3, 4)
```

Reactivo 1.2

Partiendo de algún valor inicial $x_0 \in \{1, 2, 3, 4\}$ y usando la matriz Q (función de propuesta) que aparece abajo, elabore un programa de cómputo que genere 120 valores $x_1, \dots, x_{120}$ mediante el algoritmo Metrópolis Hastings (MH)

Seleccionamos un valor inicial

```
x_0 <- sample(sop, 1)
print(x_0)
```

```
## [1] 1
```

Definimos a la matriz Q

```
Q <- matrix(0.25, nrow = 4, ncol = 4)
print(Q)
```

```
##      [,1] [,2] [,3] [,4]
## [1,] 0.25 0.25 0.25 0.25
## [2,] 0.25 0.25 0.25 0.25
## [3,] 0.25 0.25 0.25 0.25
## [4,] 0.25 0.25 0.25 0.25
```

Recordando que el algoritmo MH requiere la *probabilidad de aceptación*

$$\alpha(x_t, y) = \min \left(1, \frac{\pi(y)q(x_t|y)}{\pi(x_t)q(y|x_t)} \right)$$

Para este problema en particular veamos que para toda $i, j \in \{1, 2, 3, 4\}$,

$$q(j|i) = Q_{ij} = Q_{ji} = q(i|j)$$

Por lo que la probabilidad de aceptación depende únicamente de la distribución objetivo

$$\alpha(x_t, y) = \min \left(1, \frac{\pi(y)}{\pi(x_t)} \right)$$

Así, simulamos

```
n <- 120
x_t <- x_0
states <- c()
for (idx in 1:n) {
  y <- as.integer(sample(x = sop, size = 1, prob = obj))
  alpha_ij <- min(1, obj[y]/obj[x_t])
  u <- runif(n = 1)
  if (u < alpha_ij) {
    # Aceptación
    x_t <- y
  } # Rechazo -> x_t no se actualiza
  states <- c(states, x_t)
}
print(states)

##      [1] 1 2 3 4 2 2 4 4 4 4 4 1 3 3 3 2 3 2 4 3 3 3 3 3 3 4 2 2 4 3 3 3 4 4 4 4 3
##     [38] 2 4 4 4 4 3 4 4 4 4 2 3 3 4 4 4 4 4 3 3 4 2 3 3 4 1 2 3 3 3 4 3 4 4 2 1
##     [75] 4 4 4 2 4 4 3 4 4 4 3 3 4 4 4 2 4 4 2 4 2 4 4 4 4 3 3 3 4 3 4 4 4 2 4
##    [112] 4 2 4 4 4 4 2 2 3
```

Reactivo 1.3

Definiremos a $P = \alpha_{ij}$, entonces creamos una función llamada **alpha** para el cálculo de las probabilidades de aceptación

```
alpha <- function(i, j) {
  obj <- c(0.1, 0.2, 0.3, 0.4)
  x <- min(1, obj[j]/obj[i])
  return (x)
}
print(alpha(1,4))
```

```
## [1] 1
```

Así definimos a P como

```
P <- outer(1:4, 1:4, Vectorize(alpha))
print(P)
```

```
##           [,1]      [,2] [,3] [,4]
## [1,] 1.0000000 1.0000000 1.00    1
## [2,] 0.5000000 1.0000000 1.00    1
## [3,] 0.3333333 0.6666667 1.00    1
## [4,] 0.2500000 0.5000000 0.75    1
```

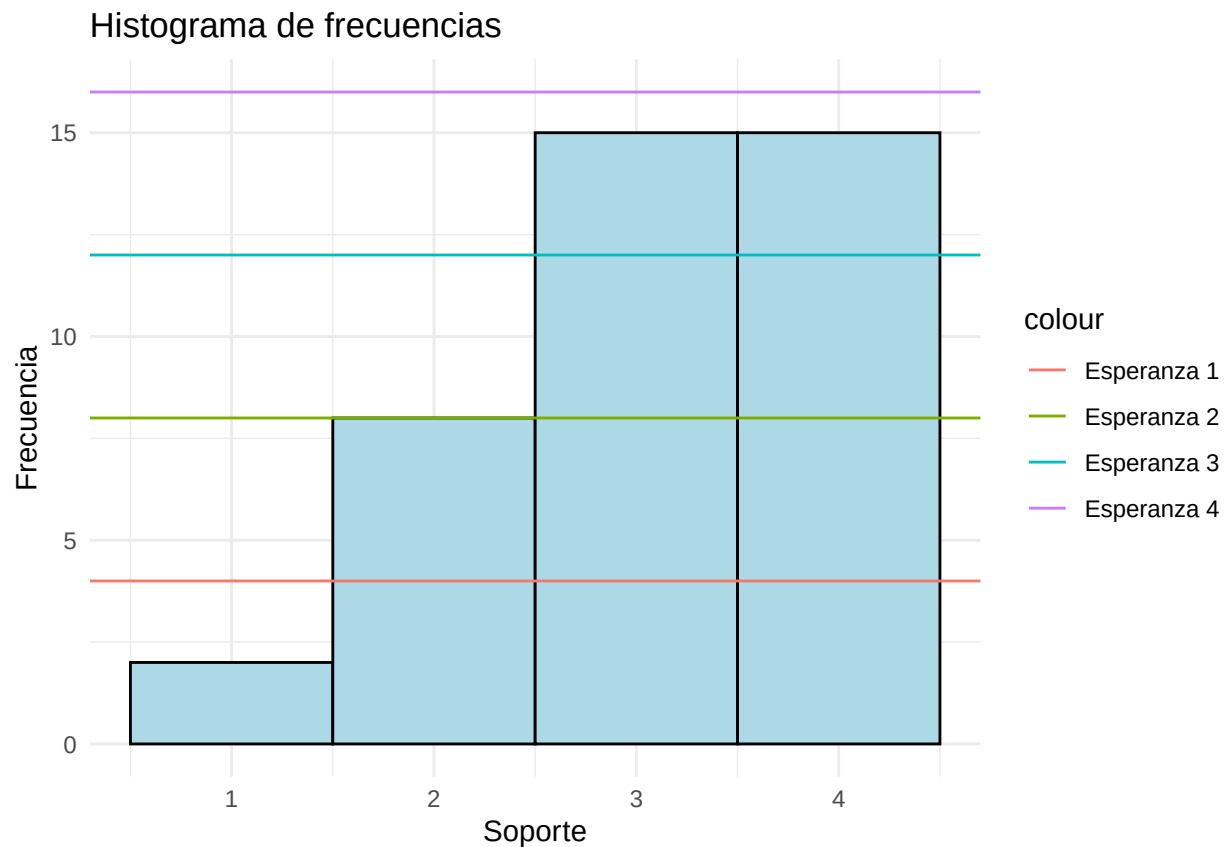
Observemos que es irreducible, ya que no hay probabilidades de transición que sean iguales a cero. De igual forma es aperiódica ni estados absorbentes. Por lo tanto la matriz es convergente.

Reactivos 1.4, 1.5, 1.6

Graficando los histogramas

```
df <- data.frame(values = states[1:40])

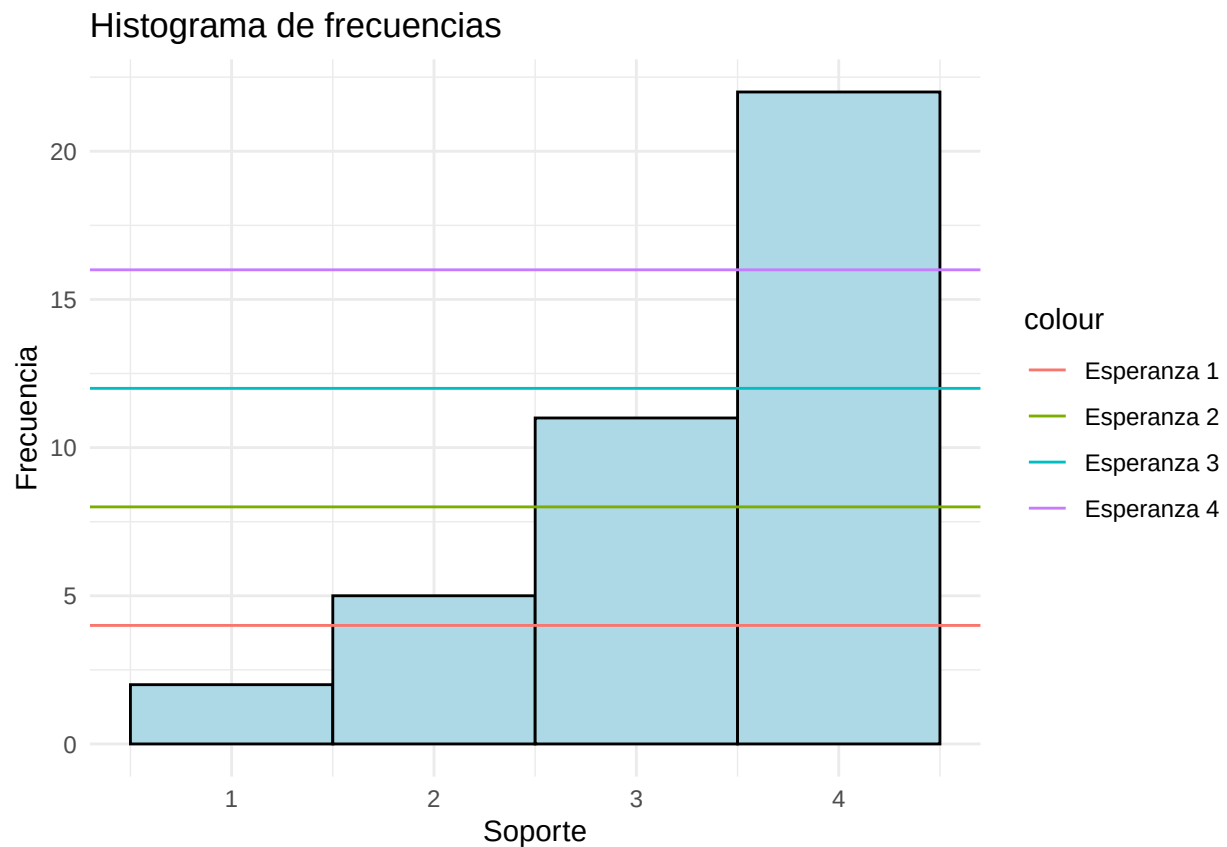
ggplot(df, aes(x = values)) +
  geom_histogram(binwidth = 1, fill = "lightblue", color = "black") +
  labs(title = "Histograma de frecuencias", x = "Soporte", y = "Frecuencia") +
  geom_hline(aes(yintercept = 40 * obj[1], color="Esperanza 1")) +
  geom_hline(aes(yintercept = 40 * obj[2], color="Esperanza 2")) +
  geom_hline(aes(yintercept = 40 * obj[3], color="Esperanza 3")) +
  geom_hline(aes(yintercept = 40 * obj[4], color="Esperanza 4")) +
  theme_minimal()
```



Para la segunda parte

```
df <- data.frame(values = states[41:80])

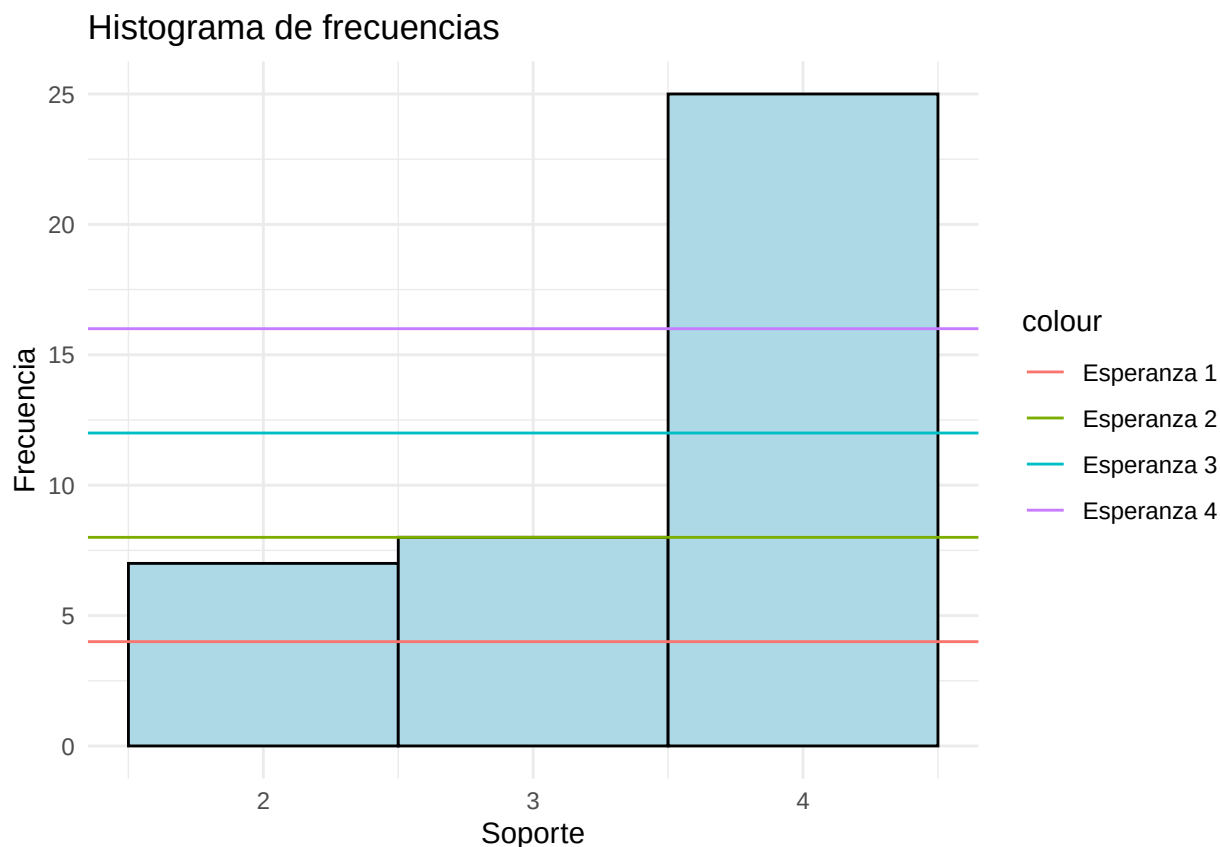
ggplot(df, aes(x = values)) +
  geom_histogram(binwidth = 1, fill = "lightblue", color = "black") +
  labs(title = "Histograma de frecuencias", x = "Soporte", y = "Frecuencia") +
  geom_hline(aes(yintercept = 40 * obj[1], color="Esperanza 1")) +
  geom_hline(aes(yintercept = 40 * obj[2], color="Esperanza 2")) +
  geom_hline(aes(yintercept = 40 * obj[3], color="Esperanza 3")) +
  geom_hline(aes(yintercept = 40 * obj[4], color="Esperanza 4")) +
  theme_minimal()
```



Para la tercera parte

```
df <- data.frame(values = states[81:120])

ggplot(df, aes(x = values)) +
  geom_histogram(binwidth = 1, fill = "lightblue", color = "black") +
  labs(title = "Histograma de frecuencias", x = "Soporte", y = "Frecuencia") +
  geom_hline(aes(yintercept = 40 * obj[1], color="Esperanza 1")) +
  geom_hline(aes(yintercept = 40 * obj[2], color="Esperanza 2")) +
  geom_hline(aes(yintercept = 40 * obj[3], color="Esperanza 3")) +
  geom_hline(aes(yintercept = 40 * obj[4], color="Esperanza 4")) +
  theme_minimal()
```



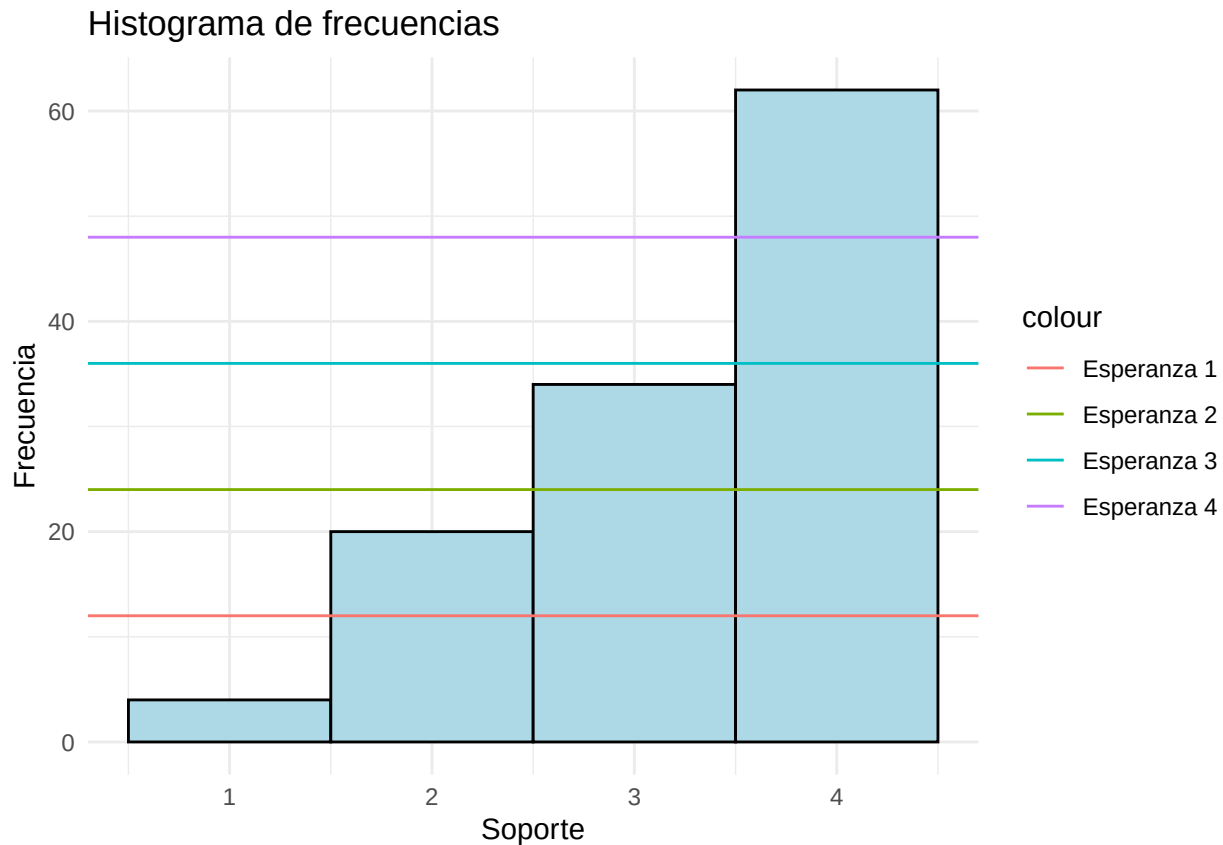
Reactivo 1.7

Podemos observar que hay una falta notoria de valores 1 en los últimos 4 elementos. Lo cual puede deberse a una para los últimos estados del soporte 1 es un elemento poco probable. Así que con base en las gráficas anteriores la última muestra de tamaño 40 **no** es mejor que las anteriores. Considero que esto debería de considerarse parte del *burn_in* y deberíamos de descartar las primeras observaciones.

Sin embargo, es importante notar que en conjunto, las muestras se distribuyen (un poco más) como esperamos:

```
df <- data.frame(values = states)

ggplot(df, aes(x = values)) +
  geom_histogram(binwidth = 1, fill = "lightblue", color = "black") +
  labs(title = "Histograma de frecuencias", x = "Soporte", y = "Frecuencia") +
  geom_hline(aes(yintercept = n * obj[1], color="Esperanza 1")) +
  geom_hline(aes(yintercept = n * obj[2], color="Esperanza 2")) +
  geom_hline(aes(yintercept = n * obj[3], color="Esperanza 3")) +
  geom_hline(aes(yintercept = n * obj[4], color="Esperanza 4")) +
  theme_minimal()
```



Problema 2

Considere el conjunto de datos Mediciones, el cual contiene registros bivariados de humedad y temperatura. Se observa que existen valores faltantes en la variable de temperatura; no obstante, se sabe por el mecanismo de recolección que los registros correspondientes a los cinco días con mayor temperatura están completos dentro del conjunto.

Construya el algoritmo EM para imputar los datos faltantes.

Cargando los datos

```
measurements <- read_csv('data/Mediciones.csv')

## Rows: 116 Columns: 2
## -- Column specification -----
## Delimiter: ","
## dbl (2): humedad, temperatura
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

head(measurements)

## # A tibble: 6 x 2
##   humedad temperatura
##   <dbl>      <dbl>
## 1   36.7       53.5
## 2    37        51
## 3   39.3       39
```

```
## 4    38.8    42
## 5    40.4    47
## 6    37.3    44.5
```

Identificamos las mediciones faltantes

```
subset(measurements, is.na(measurements$temperatura))
```

```
## # A tibble: 20 x 2
##   humedad temperatura
##   <dbl>         <dbl>
## 1    43.0          NA
## 2    37.8          NA
## 3    42.2          NA
## 4    37.8          NA
## 5    37.8          NA
## 6    41.3          NA
## 7    39.9          NA
## 8    43.4          NA
## 9    35.0          NA
## 10   36.4          NA
## 11    37           NA
## 12    39           NA
## 13   36.4          NA
## 14   38.8          NA
## 15   38.2          NA
## 16   37.8          NA
## 17   38.4          NA
## 18   38.2          NA
## 19   41.3          NA
## 20   36.4          NA
```

```
indices_faltantes <- which(is.na(measurements), arr.ind=TRUE)[,1]
print(indices_faltantes)
```

```
## [1]  8 10 16 18 19 20 28 32 36 48 55 57 62 72 78 81 101 110 112
## [20] 113
```

Vamos a estimar los valores faltantes de `temperatura` utilizando el algoritmo EM Bivariado. Definimos

```
epsilon <- 1e-5 # Tolerancia
diferencia <- 1
k <- 0 # Iteración
max_iter <- 1e3
n <- nrow(measurements)

mu_est <- c(mean(measurements$humedad), mean(measurements$temperatura, na.rm = TRUE))
sigma_est <- cov(measurements, use = "complete.obs")
print(mu_est)
```

```
## [1] 37.93103 54.55208
```

```
print(sigma_est)
```

```
##           humedad temperatura
## humedad      2.587351  -17.03004
## temperatura -17.03004  268.47094
```


Realizamos la estimación

```
while (diferencia > epsilon && k < max_iter) {
  k <- k + 1
  mu_prev <- mu_est
  sigma_prev <- sigma_est

  imputado <- measurements$temperatura
  beta_condicional <- sigma_est[1, 2] / sigma_est[1, 1]

  for(i in indices_faltantes) {
    imputado[i] <- mu_est[2] + beta_condicional * (measurements$humedad[i] - mu_est[1])
  }

  var_condicional <- sigma_est[2, 2] - (sigma_est[1, 2]^2 / sigma_est[1, 1])
  # PASO M: Maximización y actualización
  mu_est <- c(mean(measurements$humedad), mean(imputado))
  datos_imputados <- cbind(measurements$humedad, imputado)
  sigma_est <- cov(datos_imputados) * (n - 1) / n
  sigma_est[2, 2] <- sigma_est[2, 2] + ((length(indices_faltantes)/n) * var_condicional)

  diferencia <- sum((mu_est - mu_prev)^2) + sum((sigma_est - sigma_prev)^2)
}
```

Es entonces que las estimaciones de los datos faltantes son:

```
datos_imputados[indices_faltantes,]
```

```
##           imputado
## [1,] 43.05 19.64030
## [2,] 37.85 53.86651
## [3,] 42.20 25.23497
## [4,] 37.85 53.86651
## [5,] 37.85 53.86651
## [6,] 41.30 31.15873
## [7,] 39.90 40.37348
## [8,] 43.35 17.66571
## [9,] 35.05 72.29601
## [10,] 36.45 63.08126
## [11,] 37.00 59.46118
## [12,] 39.00 46.29725
## [13,] 36.45 63.08126
## [14,] 38.75 47.94274
## [15,] 38.15 51.89192
## [16,] 37.85 53.86651
## [17,] 38.45 49.91733
## [18,] 38.15 51.89192
## [19,] 41.30 31.15873
## [20,] 36.45 63.08126
```

Problema 3

Considere un portafolio con un capital inicial V_0 . Definimos el proceso de rendimientos logarítmicos continuos como $r_t = \ln(P_t/P_{t-1})$ para $t \in \{1, 2, \dots, T\}$. Suponga que los rendimientos conforman una muestra aleatoria de tamaño T proveniente de una distribución de probabilidad desconocida

F.

Dado que F no asume una forma paramétrica específica utilizaremos la *función de distribución empírica* (FDE), definida como:

$$\hat{F}_T = \frac{1}{T} \sum_{t=1}^T \mathbf{1}_{\{r_t \geq x\}}$$

Reactivo 3.a. Remuestreo

Utilizando el método de Bootstrap con B iteraciones, describa formalmente el procedimiento para generar la distribución empírica de los rendimientos simulados r^* . Explique brevemente la justificación teórica sobre la que se apoya este remuestre utilizando la FDE $\hat{F}_T$

Definimos para cada iteración $b \in \{1, \dots, B\}$ definimos la muestra

$$\{r_1^*, r_2^*, \dots, r_T^*, r_{T+b}^*\}$$

Generado con un muestreo aleatorio con reemplazo de la muestra original. Observemos además que la m.a. generada la FDE converge, para x fijo, $F_{T+b} \rightarrow F(x)$ casi seguramente. Por el teorema de Glivenko-Cantelli (Rincón & Santana, 2024) podemos afirmar que cuando $b \rightarrow \infty$

$$\sup_x |F_{T+b}(x) - F(x)| \rightarrow 0$$

Reactivo 3.b. Estimación del Valor en Riesgo (VaR)

Para un nivel de significancia $\alpha \in (0, 1)$, el Valoren Riesgo de los rendimientos, denotado como VaR_α , es el cuantil- α de la distribución. A partir de la muestra Bootstrap $\{r_1^*, r_2^*, \dots, r_B^*\}$, defina analíticamente el estimador del rendimiento crítico $\hat{r}_c$ y demuestre cómo mapear este rendimiento crítico para obtener el límite de pérdida máxima esperada en términos monetarios para el portafolio VaR^{USD}

Sea $q_\alpha(F)$ el cuantil α de la distribución F , i.e.

$$P(r_t \leq q_\alpha) = \alpha$$

Tomemos ahora a los estadísticos de orden del rendimiento. S.p.g. teniendo B elementos en nuestra m.a.

$$\{r_{(1)}^*, r_{(2)}^*, \dots, r_{(n)}^*\}$$

Entonces el rendimiento crítico $\hat{r}_c$ lo podríamos tomar como el estadístico de orden de la muestra que cumple con tener el índice $[\alpha B]$. Ésto representaría el peor rendimeinto esperado con probabilidad α . Definiéndolo con la FDE,

$$\hat{r}_c = \hat{F}_B^{-1}(\alpha)$$

Ahora, recordando que los rendimientos son logaritmicos, observamos que

$$V_{n+1} = V_n \exp(r)$$

Por lo que la pérdida sería

$$L_{n+1} = V_n - V_{n+1}$$

y particularmente, el rendimiento crítico

$$L_{c,n} = V_n - V_n \exp(\hat{r}_c) = V_n(1 - \exp(\hat{r}_c))$$

Así, el Valor en Riesgo para el capital inicial lo definimos como

$$VaR_{USD} = V_0(1 - \exp(\hat{r}_c))$$

Reactivo 3.c. Implementación Computacional

Considere las acciones de Microsoft (MSFT) y un capital $V_0 = \$1000$ USD.

Definimos el capital inicial

```
v_0 = 1000
```

Reactivo 3.c.i

Cargamos los datos en el dataframe `msft`

```
msft <- read_csv('data/MSF_close.csv')
```

```
## Rows: 262 Columns: 2
## -- Column specification -----
## Delimiter: ","
## dbl  (1): Adj_close
## date (1): Fecha
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
glimpse(msft)
```

```
## Rows: 262
## Columns: 2
## $ Fecha      <date> 2025-04-15, 2025-04-16, 2025-04-17, 2025-04-21, 2025-04-22, ~
## $ Adj_close <dbl> 382.7994, 368.7867, 364.9858, 356.3916, 364.0331, 371.5456, ~
```

Transformamos `Adj_close` en log-rendimientos

```
msft <- msft %>%
  mutate(
    r_t = log(Adj_close/lag(Adj_close))
  )
```

Eliminamos el primer valor NA

```
msft <- na.omit(msft)
```

Diferenciamos las observaciones de los valores de evaluación

```
T_est <- 252
N_val <- 10

returns_est <- msft$r_t[1:T_est]
returns_val <- msft$r_t[(T_est + 1):(T_est + N_val)]
```

Reactivo 3.c.ii

Definimos

```
B = 10000
alpha = 0.05
```

Realizamos bootstrap con reemplazo

```
bootstrap_returns <- sample(  
  x = returns_est,  
  size = B,  
  replace = TRUE  
)  
print(head(bootstrap_returns))
```

```
## [1] -0.0137365089 -0.0103422081  0.0028820429 -0.0223309252  0.0004213805  
## [6] -0.0005770833
```

Obtenemos el cuantil α para obtener el rendimiento crítico

```
r_c_hat <- quantile(  
  bootstrap_returns,  
  probs = alpha  
)  
  
r_c_hat
```

```
##          5%  
## -0.02427944
```

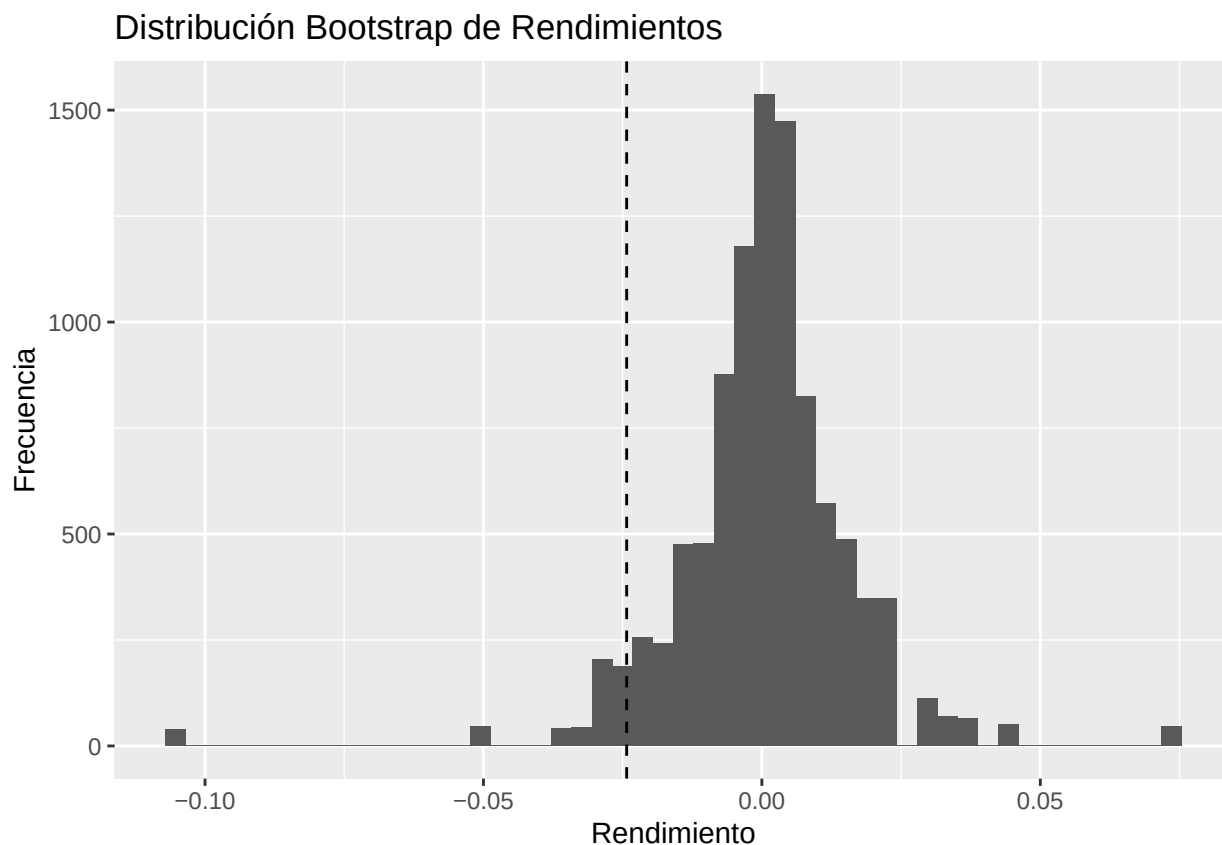
Observemos que es consistente con la clasificación AAA del activo. Calculamos el VaR

```
VaR_USD <- v_0 * (1 - exp(r_c_hat))  
  
VaR_USD
```

```
##          5%  
## 23.98707
```

Y visualizamos

```
ggplot(  
  data.frame(r_boot = bootstrap_returns),  
  aes(x = r_boot)  
) +  
  geom_histogram(  
    bins = 50  
  ) +  
  geom_vline(  
    xintercept = r_c_hat,  
    linetype = "dashed"  
  ) +  
  labs(  
    title = "Distribución Bootstrap de Rendimientos",  
    x = "Rendimiento",  
    y = "Frecuencia"  
  )
```



Reactivo 3.c.iii

Definimos el PNL como

$$\text{PnL}_t = V_0(\exp(r_t) - 1)$$

```
PnL_real <- v_0 * (exp(returns_val) - 1)
```

```
PnL_real
```

```
## [1] -11.1639375 14.5669295 20.6526068 -39.6609371 21.3349251 0.4710381
## [7] 10.4279285 -11.1590182 -39.2969727 NA
```

Creamos la indicadora de excepciones (el PNL es menor al VaR estimado) mediante un operador ternario

```
I_t <- ifelse(
  PnL_real < -VaR_USD,
  1,
  0
)
```

```
I_t
```

```
## [1] 0 0 0 1 0 0 0 0 0 1 NA
```

Sumando

```
X_obs <- sum(I_t, na.rm = TRUE)
```

```
X_obs
```

```
## [1] 2
```

Lo contrastamos contra el valor esperado

```
E_X <- N_val * alpha
```

```
E_X
```

```
## [1] 0.5
```

Y vemos que se salió de la expectativa. Esto se puede deber a la baja ventana de evaluación, que hace que se resalten las excepciones. Sería interesante contar con más datos y observar la evolución de la estimación contra el valor esperado.